Protocol Specification

The name of my protocol is Hermes.It is an application-layer protocol for messaging and file transfer between many clients and one server. It runs over TCP on port 9000 by default. Each client opens one TCP connection when it starts and keeps it open until it logs out.
TCP already provides reliable, ordered delivery, so HERMES does not repeat that work. What it adds is: a way to tell where one message ends and the next begins, a set of commands and their meanings, and rules for what a client is allowed to do at each point in a session.

Message Format

Every message is one line of JSON, ending with a single newline character (UTF 8 encoding)

```
{"op": "msg", "to": "bob", "text": "hello"}\n
```

If the JSON object has a "bytes" field, then exactly that many raw bytes follow immediately after the newline

```
{"op": "upload", "name": "notes.txt", "bytes": 1024}\n
<1024 raw bytes of file data>
```

Every message has an "op" field that says what kind of message it is.Unknown extra fields are ignored

Why i have chosen to specify the length of the message

TCP is a stream of bytes with no message boundaries in it. One call to recv() might return half a message, or two messages joined together. So the receiver has to be told where each message ends, and there are two common ways to do that: put a special marker at the end, or say the length in advance.

Because of this design choice i was able to encounter 3 things easily
-My server knows the size of the file before it arrives, so i can reject it easily if i want to
-The header is self-describing. Because it is JSON, fields have names instead of positions. list can return an actual array of usernames rather than a formatted string the client has to parse

Commands (client-server)

Message	Meaning
{"op":"login","user":u,"pass":p}	Log in; first login creates the account
{"op":"list"}	Ask who is online
{"op":"msg","to":u,"text":t}	Private message to one online user
{"op":"broadcast","text":t}	Message every other online user
{"op":"upload","name":n,"bytes":N} + N bytes	Store a file on the server
{"op":"download","name":n}	Fetch a file from the server
{"op":"sendfile","to":u,"name":n,"bytes":N} + N	Send a file to another user
{"op":"ping"}	Round-trip test
{"op":"logout"}	Disconnect

Only `login`, `ping` and `logout` are allowed before logging in. Anything else is answered with `NOT_LOGGED_IN`.

Server-Client interaction

The server sends exactly one **reply** for every command it receives:

```
{"op":"ok", ...}                  the command worked
{"op":"error","reason":REASON}    the command failed
```

Successful replies carry their result in extra fields:

```
{"op":"ok","user":"alice"}                        login
{"op":"ok","users":["alice","bob"]}               list
{"op":"ok","recipients":4}                        broadcast
{"op":"ok","stored":"notes.txt","size":1024}      upload
{"op":"ok","name":"notes.txt","bytes":1024}       download, payload follows
```

The server also sends **pushes**, which the client did not ask for:

Push	Meaning
{"op":"msg","from":u,"text":t}	Somebody messaged you privately
{"op":"broadcast","from":u,"text":t}	Somebody broadcast a message
{"op":"file","from":u,"name":n,"bytes":N} + N	Somebody sent you a file
{"op":"notice","text":t}	A user joined or left

When a client connects, before it sends anything, the server sends {"op":"hello","version":"HERMES/1.0"}.

Telling replies and pushes apart

A message from the server is a reply if its op is ok, error or hello. Anything else is a push. This one check is the whole rule, and it is what lets a single reader thread in the client handle both

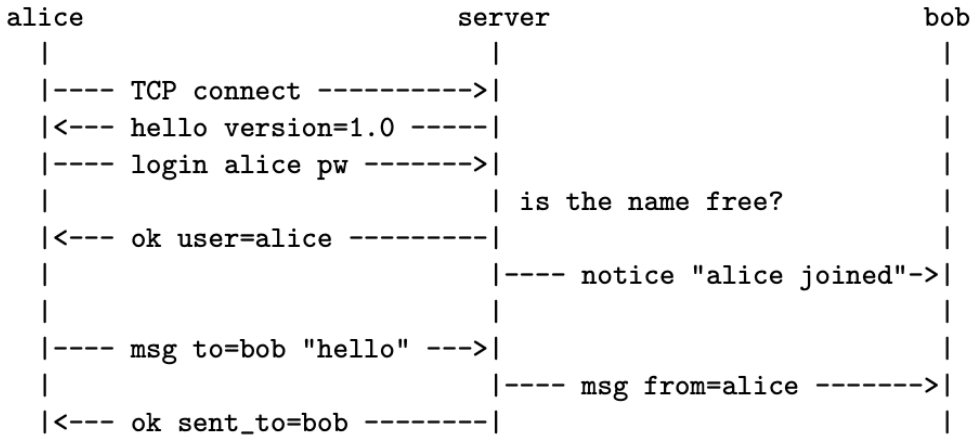
the answers to its own commands and messages arriving from other users, over one connection, without polling.

Error Messages/Bad Requests

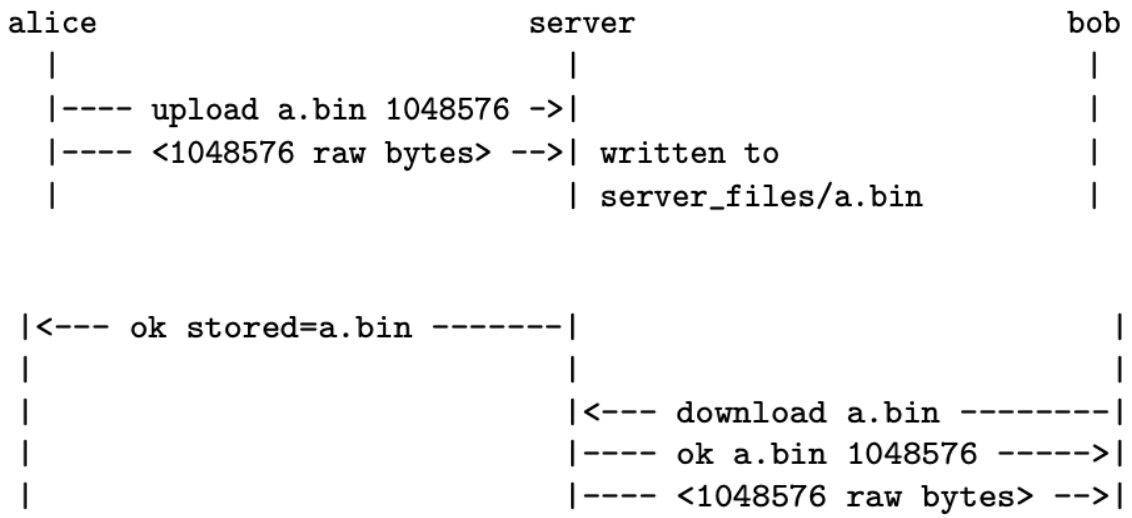
Reason	When it is sent
BAD_REQUEST	A field is missing or has an invalid value
NOT_LOGGED_IN	The command needs a logged-in user
BAD_PASSWORD	The account exists but the password is wrong
USER_ALREADY_ONLINE	Somebody is already logged in with that name
NO_SUCH_USER	The recipient is not online
NO_SUCH_FILE	The server has no file with that name
BAD_NAME	Illegal filename, for example one containing a slash
UNKNOWN_OP	The op field is not a known command

Sequence Diagrams

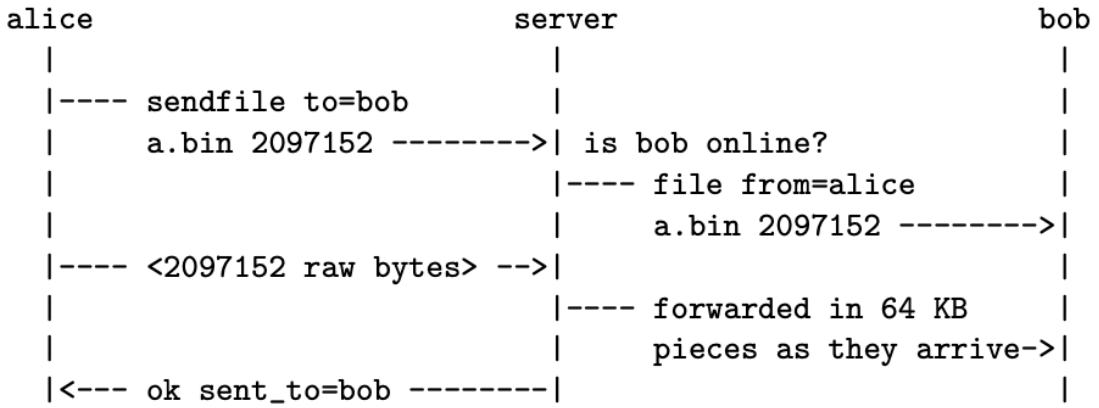
Login, then a private message



Upload, then download by another user



Client-to-client transfer through the server



The server does not store the file in this case. Each 64 KB piece is passed straight through to the recipient, so a large transfer never has to fit in the server's memory.

Rules and limits ˆ
-One TCP connection per client, open for the whole session.
-One user may be logged in from one connection at a time. A second login with the same name is refused with USER ALREADY ONLINE. ˆ
-Filenames may not contain / or \, which is what stops a client from reading or writing outside the server's server files/ folder. ˆ
- A single file may be at most 2 GB. ˆ
-Passwords are sent and stored in plain text,as per assignment guideline. We assumes a trusted network,